CS252 — Computational Thinking

Project proposal presentation

Group 03 - Seven Dwarfs

Project: SmartSchedule — Personal Productivity Assistant for Automated Task & Timetable Scheduling

Team member

Student ID	Student Name
25125028	Phạm Đức Minh
25125001	Hoàng Minh Anh
25125006	Nguyễn Nguyễn Gia Bảo
25125007	Lê Tiến Bình
25125019	Trần Nguyên Khánh
25125032	Lê Minh Nhật
25125037	Nguyễn Trọng Tiến

I. Problem Definition

Not enough time?



I. Problem Definition

You start to **schedule** your future**!**

I. Problem Definition

But… most of the time you fail to keep up with your schedule.

Why???

I. Problem Definition

You estimated finishing homework in 1h,
but it lasted 2.

You start to miss a plan, others fall along.

Your schedule only works for a machine,
not for you…

1. Root Cause Analysis

- **Root Cause 1 — Duration Bias (Planning Fallacy):**
  Systematic cognitive bias underestimating task completion times by 25–40%, ignoring possible friction in the way.



1. Root Cause Analysis

- **Root Cause 2 — Recovery Neglect:** Treating time as blocks while ignoring biological energy rhythms and protected sleep bounds.



1. Root Cause Analysis

- **Root Cause 3 — Rigid Execution:** Static timetables lack dynamic local repair mechanics; a single unexpected delay triggers a domino collapse of the weekly plan.



2. Group Thinking: Target User Domain Selection

- To establish a clear and testable problem boundary, the group split into sub-groups to debate the target user domain.

    - Group 1: General public
    - Group 2: Office workers
    - Group 3: Academic students

2. Group Thinking: Target User Domain Selection

Target User Domain	Workload and Schedule Patterns	Autonomy and Flexibility	Verdict and Rationale
General Public (Sub-group 1)	• Unstandardized • Highly unpredictable daily routines across professions	High variability	Rejected — Domain boundaries are too broad to construct testable rules.
Office Workers (Sub-group 2)			
Academic Students (Sub-group 3)			

2. Group Thinking: Target User Domain Selection

Target User Domain	Workload and Schedule Patterns	Autonomy and Flexibility	Verdict and Rationale
General Public (Sub-group 1)			
Office Workers (Sub-group 2)	• Fixed 8-hour company shifts, heavy external meeting demands	Low autonomy	Rejected — Externally controlled calendars leave little room for AI scheduling.
Academic Students (Sub-group 3)			

2. Group Thinking: Target User Domain Selection

Target User Domain	Workload and Schedule Patterns	Autonomy and Flexibility	Verdict and Rationale
General Public (Sub-group 1)			
Office Workers (Sub-group 2)			
Academic Students (Sub-group 3)	• Recurring weekly lectures • fragmented free gaps, clear deadlines	High flexibility	Accepted — Structured yet flexible domain ideal for constraint-based scheduling.

Problem Definition

"Given a **student's** fixed commitments, academic tasks with estimated durations and deadlines, and personal scheduling preferences, create a realistic weekly timetable that respects fixed commitments and protected sleep, meets deadlines when feasible, and fits preferences. If time is insufficient or constraints conflict, transparently identify conflicts and unscheduled tasks instead of claiming an impossible schedule is feasible."

3. Group Thinking: Data Abstraction Strategy

- To isolate strictly necessary parameters without user fatigue, the group split into sub-groups to evaluate input

Abstraction Level	Data Fields	Trade-off Analysis	Verdict and Rationale
High-detailed Raw Input (Sub-group 1)	Hourly energy level, homework texts	High privacy concern, heavy computational data noise	Rejected — Excessive user data friction.
Minimal To-Do List (Sub-group 2)			
Balanced Feature Set (Synthesized from Sub-groups 1 & 2)			

3. Group Thinking: Data Abstraction Strategy

- To isolate strictly necessary parameters without user fatigue, the group split into sub-groups to evaluate input

Abstraction Level	Data Fields	Trade-off Analysis	Verdict and Rationale
High-detailed Raw Input (Sub-group 1)			
Minimal To-Do List (Sub-group 2)	Task title and deadline alone	Insufficient parameters for algorithmic placement	Rejected — Missing cognitive load and splitability.
Balanced Feature Set (Synthesized from Sub-groups 1 & 2)			

3. Group Thinking: Data Abstraction Strategy

- To isolate strictly necessary parameters without user fatigue, the group split into sub-groups to evaluate input

Abstraction Level	Data Fields	Trade-off Analysis	Verdict and Rationale
High-detailed Raw Input (Sub-group 1)			
Minimal To-Do List (Sub-group 2)			
Balanced Feature Set (Synthesized from Sub-groups 1 & 2)	Core commitments, task metadata, focus preferences	Optimizes data necessity while minimizing user friction	Selected — Sufficient and minimal model.

Input and Output

INPUT

• Fixed commitments: Classes, work shifts, meetings, sleep, meals.

• Task details: Title, duration, deadline, priority, splitability.

• Preferences: Focus periods, session lengths, break needs.

OUTPUT

• Weekly time-block timetable: What, when, and how long.

• Rescheduling plan: Adjust affected tasks, preserve unaffected.

• Overload warning: Flag conflicts & unfeasible tasks.

4. Group Thinking: Computational Problem Model Selection

- To determine the underlying mathematical formulation, the group split into sub-groups to analyze problem modeling approaches.

Computational Model	Mathematical Mechanics	Trade-off Analysis	Verdict and Rationale
Pure Optimization (Sub-group 1)	Maximizes objective function (productivity / comfort scores)	May schedule work over fixed commitments or sleep if score is high	Rejected — Violates non-negotiable sleep bounds.
Pure Constraint Satisfaction (Sub-group 2)			
Constrained Optimization (Synthesized from Sub-groups 1 & 2)			

4. Group Thinking: Computational Problem Model Selection

- To determine the underlying mathematical formulation, the group split into sub-groups to analyze problem modeling approaches.

Computational Model	Mathematical Mechanics	Trade-off Analysis	Verdict and Rationale
Pure Optimization (Sub-group 1)			
Pure Constraint Satisfaction (Sub-group 2)	Strictly enforces boolean feasibility rules	Generates valid timetables but ignores energy rhythms and focus preferences	Rejected — Feasible but rigid; ignores energy preference.
Constrained Optimization (Synthesized from Sub-groups 1 & 2)			

4. Group Thinking: Computational Problem Model Selection

Computational Model	Mathematical Mechanics	Trade-off Analysis	Verdict and Rationale
Constrained Optimization (Synthesized from Sub-groups 1 & 2)		Enforces hard feasibility while optimizing focus periods and rest	Selected Hybrid Model — Optimal balance of feasibility and comfort.

Tier 1 Hard Constraints (Non-negotiable): Zero overlaps with fixed commitments (classes/work), protected sleep locking (≥ 6h/night), schedule within available free time, and meet deadlines when feasible.

Tier 2 Soft Preferences (Optimizable): Align cognitive task demand with circadian focus periods, respect session lengths and rest breaks, minimize context-switching penalties, and minimize schedule changes during repairs.

II. The Four Pillars of Computational Thinking

Decomposition, pattern recognition, abstraction, and five-step algorithmic design.



1. Group Thinking: Pillar 1 (Decomposition) Structural Strategy

To determine how to break down the sprawling scheduling problem, the group split into sub-groups to evaluate structural decomposition models.

Breakdown Approach	Focus Area	Trade-off Analysis	Verdict and Rationale
By App Screens (Sub-group 1)			
By Days of Week (Sub-group 2)			
By Decision Questions (Sub-group 3)			

1. Group Thinking: Pillar 1 (Decomposition) Structural Strategy

To determine how to break down the sprawling scheduling problem, the group split into sub-groups to evaluate structural decomposition models.

Breakdown Approach	Focus Area	Trade-off Analysis	Verdict and Rationale
By App Screens (Sub-group 1)	Calendar / To-Do / Chat UI screens	Focuses on user interface displays, failing to isolate core scheduling logic	Rejected — Black-box UI approach.
By Days of Week (Sub-group 2)			
By Decision Questions (Sub-group 3)			

1. Group Thinking: Pillar 1 (Decomposition) Structural Strategy

To determine how to break down the sprawling scheduling problem, the group split into sub-groups to evaluate structural decomposition models.

Breakdown Approach	Focus Area	Trade-off Analysis	Verdict and Rationale
By App Screens (Sub-group 1)			
By Days of Week (Sub-group 2)	Monday to Sunday time blocks	Duplicates identical scheduling logic across every single day of the week	Rejected — Repeated daily logic.
By Decision Questions (Sub-group 3)			

1. Group Thinking: Pillar 1 (Decomposition) Structural Strategy

To determine how to break down the sprawling scheduling problem, the group split into sub-groups to evaluate structural decomposition models.

Breakdown Approach	Focus Area	Trade-off Analysis	Verdict and Rationale
By App Screens (Sub-group 1)			
By Days of Week (Sub-group 2)			
By Decision Questions (Sub-group 3)	Problem-First decision layers	Exposes underlying decisions that the computational engine must solve	Selected 4-Layer Model — Problem-first architecture.

2. Pillar 1: Four-Perspective Problem Architecture and Atomic Leaves Rule

Hierarchical problem decomposition tree structured into decision layers down to concrete atomic operations.

Automated Weekly Academic Scheduler (Multi level)

Perspective 1: Problem Nature

- Free slots & task models
- Sleep bounds (≥ 6h)
- Feasibility goals

Perspective 2: Processing Logic

- Macro to micro task split
- Buffer scaling (e.g., ×1.2/×1.35)
- Multi-weight scoring

Perspective 3: System Modules

- M1 NL Parser
- M2 Grid Manager
- M3 Math Solver Engine
- M4 Local Repair Module

Perspective 4: Overload & Alerts

- Workload > Free time
- Transparent task listing
- Capacity warning alerts

Atomic Leaves Stopping Rule (Formal Criteria):

- **1. Atomic Value:** A single primitive data value (e.g., duration_min = 120).
- **2. Checkable Boolean Rule:** A deterministic truth statement (e.g., ends_before_deadline == True).
- **3. Deterministic Action:** A single non-ambiguous operation (e.g., place_task_in_slot).

3. Group Thinking: Pillar 2 (Pattern Recognition) Behavioral Classification

To find meaningful behavioral patterns in user tasks, the group split into sub-groups to debate task classification schemes.

Classification Scheme	Category Scheme	Flaws and Limitations	Verdict and Rationale
By Course Subject (Sub-group 1)			
By Cognitive Behavioral Workload (Sub-group 2)			

3. Group Thinking: Pillar 2 (Pattern Recognition) Behavioral Classification

To find meaningful behavioral patterns in user tasks, the group split into sub-groups to debate task classification schemes.

Classification Scheme	Category Scheme	Flaws and Limitations	Verdict and Rationale
By Course Subject (Sub-group 1)	Math, Physics, Literature, Electives	Subject title does not indicate duration, cognitive difficulty, or best focus time	Rejected — No scheduling value.
By Cognitive Behavioral Workload (Sub-group 2)			

3. Group Thinking: Pillar 2 (Pattern Recognition) Behavioral Classification

To find meaningful behavioral patterns in user tasks, the group split into sub-groups to debate task classification schemes.

Classification Scheme	Category Scheme	Flaws and Limitations	Verdict and Rationale
By Course Subject (Sub-group 1)			
By Cognitive Behavioral Workload (Sub-group 2)	Deep Work vs. Medium Work vs. Shallow Work	Maps directly to time-block dimensions, energy requirements, and context-switching costs	Selected — Directly computable.

4. Pillar 2: Five Human Behavioral Patterns and Deterministic Logic

Mapping recurring real-world human habits to configurable deterministic heuristic rules.

Behavioral Pattern	Real-World Human Tendency	Deterministic Logic Rule (Proposal)
Pattern 1: Planning Fallacy	Humans systematically underestimate task completion times (25–40% duration bias assumed as initial heuristic to be tested).	Apply buffer scaling: × 1.2 for light/medium tasks; × 1.35 for heavy academic tasks.
Pattern 2: Workload Types	Tasks require different focus levels (Deep, Medium, Shallow Work).	Match task cognitive load (1–3) to focus duration; cap Deep Work at 90 min max per session.
Pattern 3: Circadian Rhythms	Daily focus peaks in morning/evening and dips post-lunch (user-adjustable focus heuristics).	Align Deep Work (load 3) to Peak Energy slots (level 3); place chores in low-energy slots.
Pattern 4: Context Switching	Switching between different study subjects costs focus (10–15 min cost assumed as initial heuristic).	Batch homogeneous tasks together; add penalty when placing different task types back-to-back.
Pattern 5: Dead-Zones	Free time gaps < 30 min between fixed events are easily wasted.	Never schedule Deep Work in gaps < 30 min; assign shallow tasks or mark as active rest.

5. Group Thinking: Pillar 3 (Abstraction) Task Model and Calendar Data

To design an efficient computable task representation, the group split into sub-groups to evaluate task parameters.

Task Model	Data Attributes	Trade-off Analysis	Verdict and Rationale
Raw Detail Overload (Sub-group 1)			
Naive 3-Field Model (Sub-group 2)			
Minimal Sufficient Model (Sub-group 3)			

5. Group Thinking: Pillar 3 (Abstraction) Task Model and Calendar Data

To design an efficient computable task representation, the group split into sub-groups to evaluate task parameters.

Task Model	Data Attributes	Trade-off Analysis	Verdict and Rationale
Raw Detail Overload (Sub-group 1)	Homework text, subject, teacher name, room number, color codes	Adds computational noise and privacy risk without changing scheduling decisions	Rejected — Excessive data noise.
Naive 3-Field Model (Sub-group 2)			
Minimal Sufficient Model (Sub-group 3)			

5. Group Thinking: Pillar 3 (Abstraction) Task Model and Calendar Data

To design an efficient computable task representation, the group split into sub-groups to evaluate task parameters.

Task Model	Data Attributes	Trade-off Analysis	Verdict and Rationale
Raw Detail Overload (Sub-group 1)			
Naive 3-Field Model (Sub-group 2)	name, duration, deadline	Misses priority, cognitive load, and split capability required for optimization	Rejected — Insufficient fields.
Minimal Sufficient Model (Sub-group 3)			

5. Group Thinking: Pillar 3 (Abstraction) Task Model and Calendar Data

To design an efficient computable task representation, the group split into sub-groups to evaluate task parameters.

Task Model	Data Attributes	Trade-off Analysis	Verdict and Rationale
Raw Detail Overload (Sub-group 1)			
Naive 3-Field Model (Sub-group 2)			
Minimal Sufficient Model (Sub-group 3)	Applies Decision Test (keep facts that alter schedule placement)	Contains distinct models for fixed calendar items vs. flexible work tasks	Selected — Minimal and computable.

5. Group Thinking: Pillar 3 (Abstraction) Task Model and Calendar Data

To design an efficient computable task representation, the group split into sub-groups to evaluate task parameters.

Task Model	Data Attributes	Trade-off Analysis	Verdict and Rationale
Raw Detail Overload (Sub-group 1)			
Naive 3-Field Model (Sub-group 2)			
Minimal Sufficient Model (Sub-group 3)	Applies Decision Test (keep facts that alter schedule placement)	Contains distinct models for fixed calendar items vs. flexible work tasks	Selected — Minimal and computable.

Separation of Calendar Commitments and Flexible Tasks:

- **Fixed Commitment (Hard Fixed Items):** Hard fixed schedule items with start/end times and repetition info if present (e.g. lectures, work shifts, protected sleep).
- **Task (Flexible Work Items - 8 Core Attributes):** task_id (String), category (ASSIGNMENT/REVISION), duration_min (Integer), deadline (Timepoint), priority (1–5), is_splittable (Boolean), min_chunk_min (Integer), cognitive_load (1–3).

6. Pillar 4: Synthesized Five-Step Scheduling Algorithm Recipe

The Five-Step Synthesized Scheduling Recipe:

- Step 1: Lock Fixed Commitments and Protect Biological Sleep
- Step 2: Task Preprocessing (Buffer Scaling and Splitting)
- Step 3: Priority and Circadian Energy Placement
- Step 4: Minimal Rescheduling and Local Repair
- Step 5: Deterministic Overload Reporting





III. System Architecture, Operational Pipeline, and Resilience

Decoupled LLM parser, deterministic math solver, multi-weight heuristics, persona adaptation, and two-tier crunch mode.

1. Architecture Evolution and Decoupled Engine Design

Evolving from pure prompt-based LLM scheduling to a decoupled hybrid system for reducing scheduling errors.

Naive Single-LLM Failures

- **Arithmetic Hallucination:** LLMs struggle with exact time math, hallucinate overlapping slots, and violate hard sleep bounds.
- **Daily Tunnel Vision:** Trapped in micro-tasks; fails to plan across multi-week syllabi and assignment rubrics.
- **Energy neglect:** fails to plan schedules that consider energy levels during the day, after any tasks….

1. Architecture Evolution and Decoupled Engine Design

Evolving from pure prompt-based LLM scheduling to a decoupled hybrid system for reducing scheduling errors.

Decoupled Four-Module Engine Solution

- **Module M1 — Natural Language Parser (LLM):** Parses raw sentences, syllabi PDFs, or voice inputs into structured JSON parameters.
- **Module M2 — Constraint and Grid Manager:** Maintains time grid states, locks fixed commitments and protected sleep bounds.
- **Module M3 — Deterministic Engine (Math Solver):** Evaluates hard constraints first, then runs multi-weight scoring functions to place tasks into eligible slots.
- **Module M4 — Dynamic Local Repair Module:** Handles disruptions by shifting only conflicted blocks while preserving unaffected schedule blocks.

2. Four-Step Operational Pipeline and Multi-Weight Scheduling Heuristics

Systematic operational workflow from natural language parsing to progress tracking and local rescheduling.

The Four-Step Operational Pipeline:

- **Step 1: Decompose and Tag:** Parses raw inputs/documents, splits macro-tasks, and tags cognitive load and context.

- **Step 2: Capacity Mapping:** Locks fixed commitments and protected sleep; maps personal circadian energy peaks to free slots.

- **Step 3: Multi-Weight Scheduling:** Checks hard constraints first, then applies heuristic scoring function to place tasks into highest-scoring eligible slots and inject rest buffers.

- **Step 4: Progress Tracking and Local Rescheduling:** Captures real-time progress updates (user updates remaining duration on tasks) and triggers local rescheduling when delays occur.

2. Four-Step Operational Pipeline and Multi-Weight Scheduling Heuristics

A proposed / sample scoring function

Multi-Weight Objective Scoring Function:

Score = (w1 × Urgency) + (w2 × Importance) + (w3 × Energy Fit) - (w4 × Context Switching Penalty)

(Note: Objective weights are proposed design parameters to be evaluated and calibrated during testing).

IV. Edge Cases, Design Alternatives, and Trade-offs

System boundaries, ten real-world edge cases, four failure scenario lessons, and six core architectural trade-offs.

1. Group Thinking: System Boundaries and Non-Goals Definition

System Aspect	Boundary Scope	Detailed Coverage and Rationale
In-Scope Boundaries (Sub-groups 1 & 2)	Weekly Academic Scheduling	Academic task allocation for individual students (K-12 and University), deadline management, fixed commitments, protected sleep locking, and dynamic disruption recovery.
Out-of-Scope Non-Goals (Sub-group 3)	Personal Life Over-automation	Modifying sleep habits, health/therapy counseling, managing multi-user or team calendars, and completing academic tasks on student's behalf.

2. Group Thinking: Ten Real-World Edge Cases and Recovery Protocols

Ten Real-World Edge Cases and System Behaviors:

- **1. Incomplete Schedule Info:** Warns student that schedule reliability depends on completeness; supports recurring defaults.
- **2. Ambiguous Importance:** Uses user-defined priority with optional context; avoids definitive classification based on title alone.
- **3. Inaccurate Duration Estimates:** Accepts user-updated remaining duration in MVP to dynamically replan downstream blocks (auto-learning belongs to future Stage 4).
- **4. Excessive Workload (Infeasible):** Preserves hard sleep/class bounds and transparently lists unscheduled tasks rather than generating fake schedules.
- **5. Conflicting Deadlines:** Checks mathematical feasibility first; applies multi-weight priority scoring when not all deadlines fit.
- **6. Unsplittable Large Task:** Requires continuous free block equal to full duration; flags conflict immediately if no continuous slot exists.
- **7. Sudden Interruptions:** Freezes completed past blocks and runs local repair on remaining unexecuted tasks.
- **8. Velocity Differences:** In MVP, allows manual estimate edits; historical completion velocity learning is deferred to Stage 4.
- **9. Multi-Week Task Span:** Focuses on current-week milestone allocation while maintaining visibility of long-term deadline.
- **10. Missing Metadata:** Assigns temporary fallback defaults (duration = 60 min, priority = 3); flags missing key info for user confirmation rather than assuming defaults are accurate.

3. Group Thinking: Detailed Analysis of Four Failure Scenarios and Design Lessons

Four Failure Scenarios and Architectural Design Lessons:

- Scenario 1: Subjective Priority Ambiguity (TikTok Research vs. Entertainment)
- Scenario 2: Incorrect Duration Estimation (2h Task Requires 5h)
- Scenario 3: Insufficient Available Time (7h Work vs. 6h Free)
- Scenario 4: Disruption Recovery (Unexpected Meeting Overlap)





4. Group Thinking: Six Core Design Trade-offs Matrix

Design Decision	Alternative A	Alternative B	Chosen Strategy and Rationale
1. Priority (Sub-group 1)	Infer priority automatically via AI	Require user-entered priority	User-defined priority with optional AI suggestions (Avoids misclassification).
2. Data Depth (Sub-group 2)	Require detailed daily logs	Collect minimal essential inputs	Minimal required inputs with progressive updates (Reduces user friction).
3. Duration Est. (Sub-group 3)	Predict duration from description	Request user estimates + feedback	Initial user estimates with manual progress updates in MVP (Velocity auto-learning deferred to Stage 4).
4. Disruption (Sub-group 1)	Regenerate full weekly schedule	Modify only affected blocks	Localized rescheduling (Target Schedule Stability ≥ 90%; preserves user mental map).
5. Horizon (Sub-group 2)	Long-term multi-month goals	Weekly concrete tasks	Focus on weekly horizon first; extend to multi-month milestones later.
6. Capacity (Sub-group 3)	Uniform capacity for all users	Individual preferences and rules	Persona-adapted focus windows and session lengths (K-12 and University MVP).

V. Evaluation Methodology, Metrics, and Future Roadmap

Five core research questions, scenario-based baseline testing, quantitative benchmarks, and 5-stage roadmap.

1. Group Thinking: Five Core Research Questions and Evaluation Framework

To rigorously measure system efficacy, the group split into sub-groups to establish five research questions and evaluation criteria.

Five Core Research Questions (EQ1 to EQ5):

- **EQ1 — Schedule Correctness:** Can the system generate schedules without violating hard constraints (zero overlaps with fixed commitments or protected sleep)?
- **EQ2 — Scheduling Effectiveness:** Can the system allocate all tasks before their deadlines when a mathematically feasible schedule exists?
- **EQ3 — Rescheduling Stability:** Can the system recover from unexpected disruptions without unnecessarily changing unrelated scheduled activities (target ≥ 90% stability)?
- **EQ4 — User Experience:** Does the assistant significantly reduce manual re-planning effort while providing understandable and trustworthy recommendations?
- **EQ5 — Robustness:** How does the system behave when inputs are incomplete, ambiguous, inaccurate, or mathematically infeasible?

2. Group Thinking: Scenario-Based and Comparative Testing Methodology

To validate system performance, the group split into sub-groups to design testing scenarios and baseline comparisons.

Testing Methodology and Baseline Comparisons:

- **Controlled Scenario-Based Testing (T01–T10):** Evaluates normal cases, deadline overlaps, task splitting, event disruptions, duration overruns, and infeasible workload warnings. Expected outcomes will be manually verified or checked using an independent solver for small instances, where available and planned.

Comparative Testing Framework:

- **Baseline A (Manual Scheduling):** Users construct and repair timetables manually using basic calendars -> Measures user effort and re-planning friction.
- **Baseline B (Simple Rule-Based - EDF):** Earliest Deadline First greedy placement -> Evaluates whether multi-weight optimization provides real benefits.
- **Proposed Approach (Constraint-Based Adaptive):** Evaluates full system balancing hard constraints, energy fit, buffers, and local repair stability.

3. Quantitative Evaluation Metrics and Target Benchmarks

Proposed quantitative evaluation metrics and performance target goals.

Evaluation Metric	Measurement Description	Proposed Target Benchmark
Hard Constraint Violations	Number of overlaps with commitments or protected sleep	Zero violations (0)
Deadline Satisfaction Rate	Percentage of tasks completed before deadline in feasible cases	100% for feasible cases
Infeasibility Accuracy	Correct classification of feasible vs. infeasible workloads	100% accuracy
Conflict Coverage	Percentage of unplaced tasks or deadline conflicts reported	100% coverage
Schedule Stability Metric	Percentage of unaffected blocks preserved during local repair	Target >= 90%
Execution Time	Computation time to generate or repair a 50-task weekly plan	< 2.0 seconds
Manual Planning Effort	Time / operations required to create or update weekly schedules	> 50% reduction vs Baseline A

(Note: All numerical targets are proposed evaluation goals, not achieved results. Runtime targets apply to standard execution environments for a 50-task weekly plan).

4. Group Thinking: Usability Testing, System Limitations, and Five-Stage Roadmap

To evaluate human factors and plan future enhancements, the group split into sub-groups to outline usability testing, limitations, and roadmap.

Usability Testing & System Limitations

- **Exploratory Usability Pilot Study:** 8–12 university students using 5-point Likert scale questionnaires assessing Ease of Use, Schedule Quality, Personalization, Transparency, Adaptability, and User Control.
- **Key System Limitations:** Dependence on complete user commitments; subjective importance ambiguity; task duration uncertainty; cold-start challenge for new users.

Five-Stage Development Roadmap

- **Stage 1 — Core Scheduling Engine:** Task input, fixed commitments, deadlines, buffer scaling, and feasibility checking.
- **Stage 2 — Adaptive Rescheduling:** Task splitting, disruption handling, and local repair mechanics (Target Stability ≥ 90%).
- **Stage 3 — Evaluation and Usability:** Scenario benchmarks, baseline comparative testing, and student feedback collection.
- **Stage 4 — Personalization (Future Work):** Velocity learning from historical completion data to auto-calibrate duration buffers.
- **Stage 5 — Long-Term Goal Planning (Future Work):** Decomposing multi-month macro goals into weekly schedulable milestones.

THANKS FOR LISTENING

